BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Practical No: 5

Division: C Batch: 2

Roll No: 3090319 PRN No: 12410562

Name of Student: Parth Kahane

Subject Name: Deep Learning

Problem Statement:

Implement and compare RNN, LSTM, and GRU models for sequence classification, and analyze their performance using appropriate evaluation metrics.

Algorithm :

Dataset Description
The **Reuters Newswire dataset** contains news articles categorized into **46 different topics**. Each news article is represented as a sequence of integer word IDs. The objective is to predict the category/topic of the news article.
For example, categories may represent topics related to:

- Economics
- Trade
- Politics
- Business
- Commodities
- Finance
- etc.

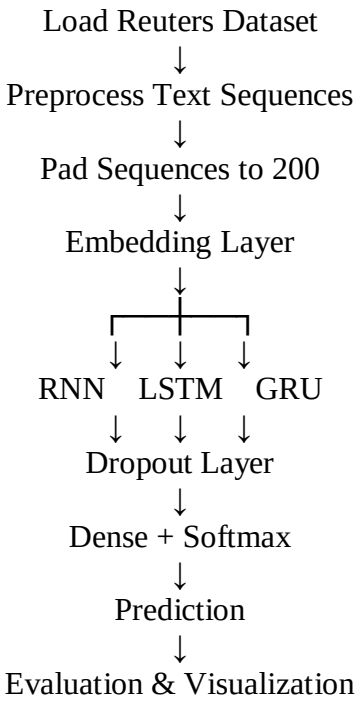
The dataset is suitable for comparing recurrent neural network architectures because the input is sequential text data.

RNN
RNNs are designed for sequential data. They maintain a hidden state that carries information from previous time steps. However, standard RNNs can suffer from the vanishing-gradient problem, making it difficult to learn long-term dependencies.

LSTM
LSTM is an improved RNN architecture that uses memory cells and gates to retain important information for longer periods. It contains forget, input, and output gates.

GRU
GRU is a simpler gated recurrent architecture. It uses update and reset gates to control information flow and can handle long-term dependencies with fewer parameters than LSTM in many configurations.

Methodology:



Evaluation Metrics

- **Accuracy:** Percentage of correctly classified samples.
- **Precision:** Correct positive predictions among all positive predictions.
- **Recall:** Correctly identified samples among all actual samples.
- **F1-score:** Balance between precision and recall.
- **Confusion Matrix:** Shows correct and incorrect predictions for each class.
- **Training Time:** Measures computational efficiency.

Results:

Model	Accuracy	Precision	Recall	F1
RNN	50.76%	41.20%	50.76%	42.23%
LSTM	55.08%	50.68%	55.08%	**50.96%**
GRU	**56.14%**	48.75%	**56.14%**	50.72%

Code :

```
# RNN vs LSTM vs GRU
# Reuters Newswire Sequence Classification
# 1. INSTALL / IMPORT LIBRARIES

!pip -q install scikit-learn

import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import time
import gc

from tensorflow.keras.datasets import reuters
from tensorflow.keras.preprocessing.sequence import pad_sequences

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, LSTM, GRU, Dense,
Dropout

from tensorflow.keras.callbacks import EarlyStopping

from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    confusion_matrix,
    classification_report
)
```

```
# 2. SET RANDOM SEED

SEED = 42

np.random.seed(SEED)
tf.random.set_seed(SEED)


# 3. PARAMETERS

MAX_FEATURES = 10000
MAX_LEN = 200

EMBEDDING_DIM = 128
HIDDEN_UNITS = 64

EPOCHS = 5
BATCH_SIZE = 128

NUM_CLASSES = 46


# 4. LOAD REUTERS DATASET

print("Loading Reuters dataset...")

(x_train, y_train), (x_test, y_test) = reuters.load_data(
    num_words=MAX_FEATURES
)


# 5. DISPLAY DATASET INFORMATION

print("\n================ DATASET INFORMATION ================")

print("Training samples :", len(x_train))
print("Testing samples  :", len(x_test))
print("Number of classes:", NUM_CLASSES)

print("\nFirst training sample:")
print(x_train[0][:20])

print("\nFirst training label:")
print(y_train[0])
```

```python
# 6. CHECK SEQUENCE LENGTHS

train_lengths = [len(x) for x in x_train]

print("\n================ SEQUENCE INFORMATION ================")

print("Minimum sequence length :", min(train_lengths))
print("Maximum sequence length :", max(train_lengths))
print("Average sequence length :", round(np.mean(train_lengths), 2))
print("Median sequence length  :", np.median(train_lengths))


# 7. VISUALIZE SEQUENCE LENGTH DISTRIBUTION

plt.figure(figsize=(10, 5))

plt.hist(
    train_lengths,
    bins=50,
    edgecolor='black'
)

plt.axvline(
    MAX_LEN,
    linestyle='--',
    linewidth=2,
    label=f'MAX_LEN = {MAX_LEN}'
)

plt.title("Distribution of Reuters Sequence Lengths")
plt.xlabel("Sequence Length")
plt.ylabel("Number of Articles")
plt.legend()
plt.show()


# 8. VISUALIZE CLASS DISTRIBUTION

class_counts = np.bincount(
    y_train,
    minlength=NUM_CLASSES
)

plt.figure(figsize=(14, 5))
```

```
plt.bar(
    range(NUM_CLASSES),
    class_counts
)

plt.title("Reuters Training Dataset - Class Distribution")
plt.xlabel("Class / Topic ID")
plt.ylabel("Number of Articles")

plt.xticks(range(NUM_CLASSES))

plt.show()


# 9. PAD SEQUENCES

print("\nPadding sequences...")

x_train = pad_sequences(
    x_train,
    maxlen=MAX_LEN
)

x_test = pad_sequences(
    x_test,
    maxlen=MAX_LEN
)

print("Training shape:", x_train.shape)
print("Testing shape :", x_test.shape)


# 10. BUILD MODEL

def build_model(kind):

    model = Sequential()



    # mask_zero ignores padding tokens

    model.add(

        Embedding(
```

```
            input_dim=MAX_FEATURES,
            output_dim=128,
            input_length=MAX_LEN,
            mask_zero=True
        )
    )

    if kind == "RNN":
        model.add(SimpleRNN(128))
    elif kind == "LSTM":
        model.add(LSTM(128))
    elif kind == "GRU":
        model.add(GRU(128))

    model.add(Dropout(0.3))
    model.add(Dense(64, activation="relu"))
    model.add(Dropout(0.2))
    model.add(Dense(NUM_CLASSES, activation="softmax"))

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
```

```
    return model

# 11. TRAIN AND EVALUATE MODELS

results = []

histories = {}

predictions = {}

confusion_matrices = {}

training_times = {}

models = {}


for kind in ["RNN", "LSTM", "GRU"]:

    print("\n")
    print("=" * 70)
    print("TRAINING:", kind)
    print("=" * 70)

    # Build model
    model = build_model(kind)

    print("\nModel Summary:")
    model.summary()

    # Early stopping
    early_stopping = EarlyStopping(
        monitor="val_loss",
        patience=2,
        restore_best_weights=True
    )

    # Start timer
    start_time = time.time()

    # Train model
    history = model.fit(
        x_train,
        y_train,
        epochs=EPOCHS,
```

```
        batch_size=BATCH_SIZE,
        validation_split=0.2,
        callbacks=[early_stopping],
        verbose=1
    )

    # End timer
    training_time = time.time() - start_time

    training_times[kind] = training_time

    # Store history
    histories[kind] = history.history

    # Prediction

    pred_prob = model.predict(
        x_test,
        verbose=0
    )

    # Convert probabilities to class labels
    pred = np.argmax(
        pred_prob,
        axis=1
    )

    predictions[kind] = pred

    # Evaluation Metrics

    accuracy = accuracy_score(
        y_test,
        pred
    )

    precision, recall, f1, _ = precision_recall_fscore_support(
        y_test,
        pred,
        average="weighted",
        zero_division=0
    )

    # Confusion matrix
    cm = confusion_matrix(
```

```
        y_test,
        pred
    )

    confusion_matrices[kind] = cm

    # Store model
    models[kind] = model

    # Number of parameters
    parameters = model.count_params()

    # Store results
    results.append([
        kind,
        accuracy,
        precision,
        recall,
        f1,
        training_time,
        parameters
    ])

    # Print metrics
    print("\n", kind, "RESULTS")

    print("Accuracy :", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall   :", round(recall, 4))
    print("F1 Score :", round(f1, 4))

    print("Training Time:",
          round(training_time, 2),
          "seconds")

    print("Parameters:",
          parameters)

    # Classification report
    print("\nClassification Report:")

    print(
        classification_report(
            y_test,
            pred,
```

```
            zero_division=0
        )
    )


# 12. CREATE FINAL RESULTS DATAFRAME

df = pd.DataFrame(
    results,
    columns=[
        "Model",
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score",
        "Training Time (sec)",
        "Parameters"
    ]
)
print("\n")
print("=" * 70)
print("FINAL MODEL COMPARISON")
print("=" * 70)

print(df)
# 13. ROUND RESULTS FOR BETTER DISPLAY
display_df = df.copy()

display_df["Accuracy"] = display_df["Accuracy"].round(4)
display_df["Precision"] = display_df["Precision"].round(4)
display_df["Recall"] = display_df["Recall"].round(4)
display_df["F1 Score"] = display_df["F1 Score"].round(4)
display_df["Training Time (sec)"] = \
    display_df["Training Time (sec)"].round(2)

print("\nFormatted Results:")

display(display_df)


# 14. TRAINING ACCURACY COMPARISON

plt.figure(figsize=(12, 6))

for kind in ["RNN", "LSTM", "GRU"]:
```

```
    plt.plot(
        histories[kind]["accuracy"],
        marker="o",
        label=kind
    )

plt.title("Training Accuracy Comparison")

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.legend()

plt.grid(True)

plt.show()


# 15. VALIDATION ACCURACY COMPARISON

plt.figure(figsize=(12, 6))

for kind in ["RNN", "LSTM", "GRU"]:

    plt.plot(
        histories[kind]["val_accuracy"],
        marker="o",
        label=kind
    )

plt.title("Validation Accuracy Comparison")

plt.xlabel("Epoch")

plt.ylabel("Validation Accuracy")

plt.legend()

plt.grid(True)

plt.show()

# 16. TRAINING LOSS COMPARISON
```

```
plt.figure(figsize=(12, 6))

for kind in ["RNN", "LSTM", "GRU"]:

    plt.plot(
        histories[kind]["loss"],
        marker="o",
        label=kind
    )

plt.title("Training Loss Comparison")

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.legend()

plt.grid(True)

plt.show()


# 17. VALIDATION LOSS COMPARISON

plt.figure(figsize=(12, 6))

for kind in ["RNN", "LSTM", "GRU"]:

    plt.plot(
        histories[kind]["val_loss"],
        marker="o",
        label=kind
    )

plt.title("Validation Loss Comparison")

plt.xlabel("Epoch")

plt.ylabel("Validation Loss")

plt.legend()

plt.grid(True)
```

```
plt.show()


# 18. METRIC COMPARISON
metrics = [
    "Accuracy",
    "Precision",
    "Recall",
    "F1 Score"
]

plot_df = df.set_index("Model")[metrics]

plot_df.plot(
    kind="bar",
    figsize=(12, 6)
)

plt.title("RNN vs LSTM vs GRU - Performance Comparison")

plt.xlabel("Model")

plt.ylabel("Score")

plt.ylim(0, 1)

plt.xticks(rotation=0)

plt.grid(axis="y")

plt.legend()

plt.show()


# 19. INDIVIDUAL METRIC BAR CHARTS

for metric in metrics:

    plt.figure(figsize=(8, 5))

    plt.bar(
        df["Model"],
        df[metric]
```

```
    )

    plt.title(
        metric + " Comparison"
    )

    plt.xlabel("Model")

    plt.ylabel(metric)

    plt.ylim(0, 1)

    plt.grid(
        axis="y"
    )

    plt.show()


# 20. TRAINING TIME COMPARISON
plt.figure(figsize=(8, 5))

plt.bar(
    df["Model"],
    df["Training Time (sec)"]
)

plt.title("Training Time Comparison")

plt.xlabel("Model")

plt.ylabel("Training Time (seconds)")

plt.grid(
    axis="y"
)

plt.show()


# 21. PARAMETER COUNT COMPARISON

plt.figure(figsize=(8, 5))

plt.bar(
```

```
    df["Model"],
    df["Parameters"]
)

plt.title("Model Parameter Comparison")

plt.xlabel("Model")

plt.ylabel("Number of Parameters")

plt.grid(
    axis="y"
)

plt.show()


# 22. ACTUAL VS PREDICTED CLASS DISTRIBUTION
actual_counts = np.bincount(
    y_test,
    minlength=NUM_CLASSES
)

plt.figure(figsize=(14, 5))

plt.plot(
    range(NUM_CLASSES),
    actual_counts,
    marker="o",
    label="Actual"
)

for kind in ["RNN", "LSTM", "GRU"]:

    predicted_counts = np.bincount(
        predictions[kind],
        minlength=NUM_CLASSES
    )

    plt.plot(
        range(NUM_CLASSES),
        predicted_counts,
        marker=".",
        label=kind
    )
```

```
plt.title(
    "Actual vs Predicted Class Distribution"
)

plt.xlabel("Class")

plt.ylabel("Number of Samples")

plt.legend()

plt.grid(True)

plt.show()

# 23. FIND BEST MODEL FOR EACH METRIC
print("\n")
print("=" * 70)
print("BEST PERFORMANCE BY METRIC")
print("=" * 70)

for metric in metrics:

    best_index = df[metric].idxmax()

    best_model = df.loc[
        best_index,
        "Model"
    ]

    best_score = df.loc[
        best_index,
        metric
    ]

    print(
        f"{metric}: {best_model} "
        f"({best_score:.4f})"
    )


# 24. FREE MEMORY
gc.collect()

print("\nExperiment completed successfully.")
```

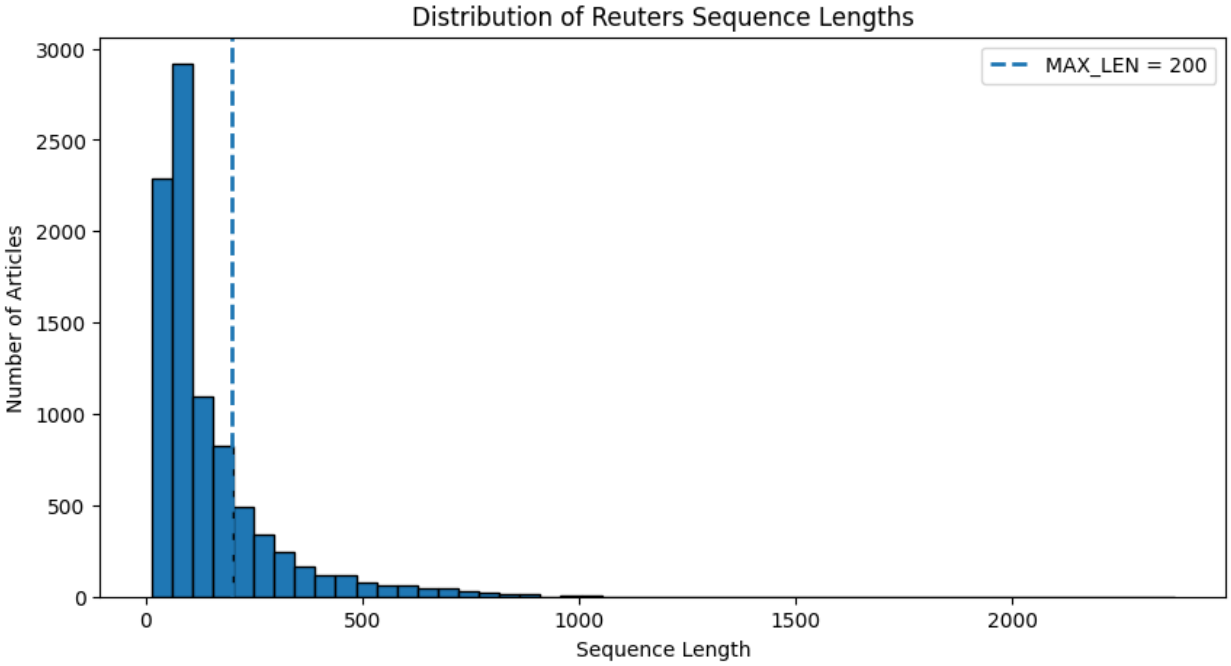
Output:
Loading Reuters dataset...

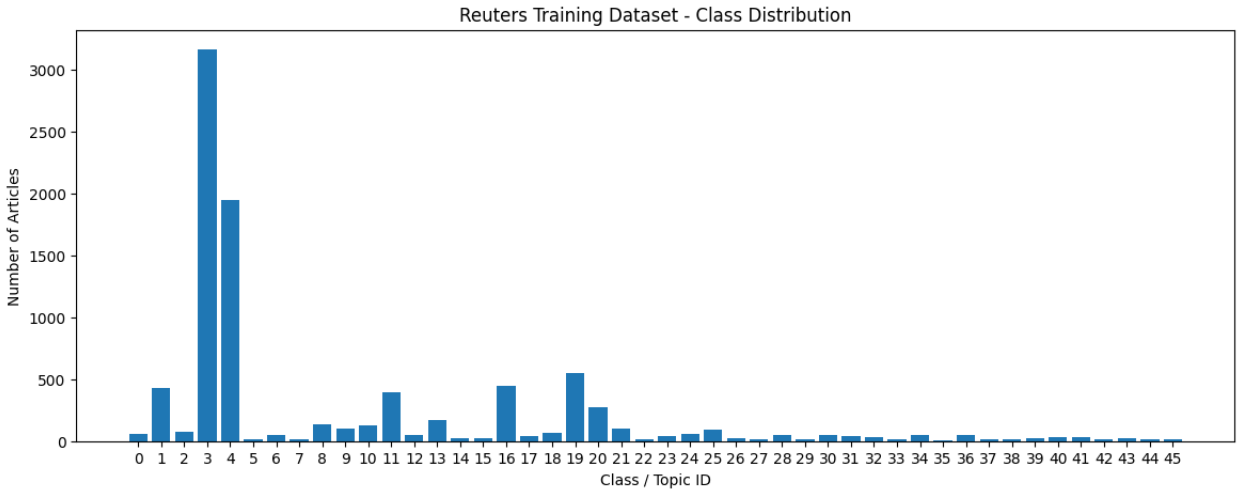
========== DATASET INFORMATION ==========
Training samples : 8982
Testing samples : 2246
Number of classes: 46

First training sample:
[1, 2, 2, 8, 43, 10, 447, 5, 25, 207, 270, 5, 3095, 111, 16, 369, 186, 90, 67, 7]

First training label:
3

========== SEQUENCE INFORMATION ==========
Minimum sequence length : 13
Maximum sequence length : 2376
Average sequence length : 145.54
Median sequence length : 95.0





Padding sequences...
Training shape: (8982, 200)
Testing shape : (2246, 200)

==========
TRAINING: RNN
==========

Model Summary:
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
warnings.warn(
Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	0 (unbuilt)
simple_rnn (SimpleRNN)	?	0 (unbuilt)
dropout_2 (Dropout)	?	0
dense_2 (Dense)	?	0 (unbuilt)
dropout_3 (Dropout)	?	0

dense_3 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

Epoch 1/5
57/57 ━━━━━━━━━━━━━━━━━━━━ 24s 388ms/step - accuracy: 0.3076 - loss: 2.7554 - val_accuracy: 0.2220 - val_loss: 2.4720
Epoch 2/5
57/57 ━━━━━━━━━━━━━━━━━━━━ 20s 356ms/step - accuracy: 0.3382 - loss: 2.4755 - val_accuracy: 0.4090 - val_loss: 2.3159
Epoch 3/5
57/57 ━━━━━━━━━━━━━━━━━━━━ 22s 382ms/step - accuracy: 0.4058 - loss: 2.2274 - val_accuracy: 0.4430 - val_loss: 2.1307
Epoch 4/5
57/57 ━━━━━━━━━━━━━━━━━━━━ 22s 378ms/step - accuracy: 0.5100 - loss: 1.8709 - val_accuracy: 0.4469 - val_loss: 2.0422
Epoch 5/5
57/57 ━━━━━━━━━━━━━━━━━━━━ 21s 366ms/step - accuracy: 0.5951 - loss: 1.5459 - val_accuracy: 0.4591 - val_loss: 2.0511

RNN RESULTS
Accuracy : 0.4452
Precision: 0.4126
Recall : 0.4452
F1 Score : 0.3993
Training Time: 128.94 seconds
Parameters: 1324142

Classification Report:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	12
1	0.50	0.03	0.05	105
2	0.00	0.00	0.00	20
3	0.79	0.69	0.74	813
4	0.41	0.67	0.51	474
5	0.00	0.00	0.00	5
6	0.00	0.00	0.00	14
7	0.00	0.00	0.00	3
8	0.00	0.00	0.00	38
9	0.00	0.00	0.00	25
10	0.00	0.00	0.00	30
11	0.00	0.00	0.00	83
12	0.00	0.00	0.00	13

13	0.00	0.00	0.00	37
14	0.00	0.00	0.00	2
15	0.00	0.00	0.00	9
16	0.13	0.15	0.14	99
17	0.00	0.00	0.00	12
18	0.00	0.00	0.00	20
19	0.17	0.79	0.28	133
20	0.00	0.00	0.00	70
21	0.00	0.00	0.00	27
22	0.00	0.00	0.00	7
23	0.00	0.00	0.00	12
24	0.00	0.00	0.00	19
25	0.00	0.00	0.00	31
26	0.00	0.00	0.00	8
27	0.00	0.00	0.00	4
28	0.00	0.00	0.00	10
29	0.00	0.00	0.00	4
30	0.00	0.00	0.00	12
31	0.00	0.00	0.00	13
32	0.00	0.00	0.00	10
33	0.00	0.00	0.00	5
34	0.00	0.00	0.00	7
35	0.00	0.00	0.00	6
36	0.00	0.00	0.00	11
37	0.00	0.00	0.00	2
38	0.00	0.00	0.00	3
39	0.00	0.00	0.00	5
40	0.00	0.00	0.00	10
41	0.00	0.00	0.00	8
42	0.00	0.00	0.00	3
43	0.00	0.00	0.00	6
44	0.00	0.00	0.00	5
45	0.00	0.00	0.00	1
accuracy			0.45	2246
macro avg	0.04	0.05	0.04	2246
weighted avg	0.41	0.45	0.40	2246

==========

TRAINING: LSTM

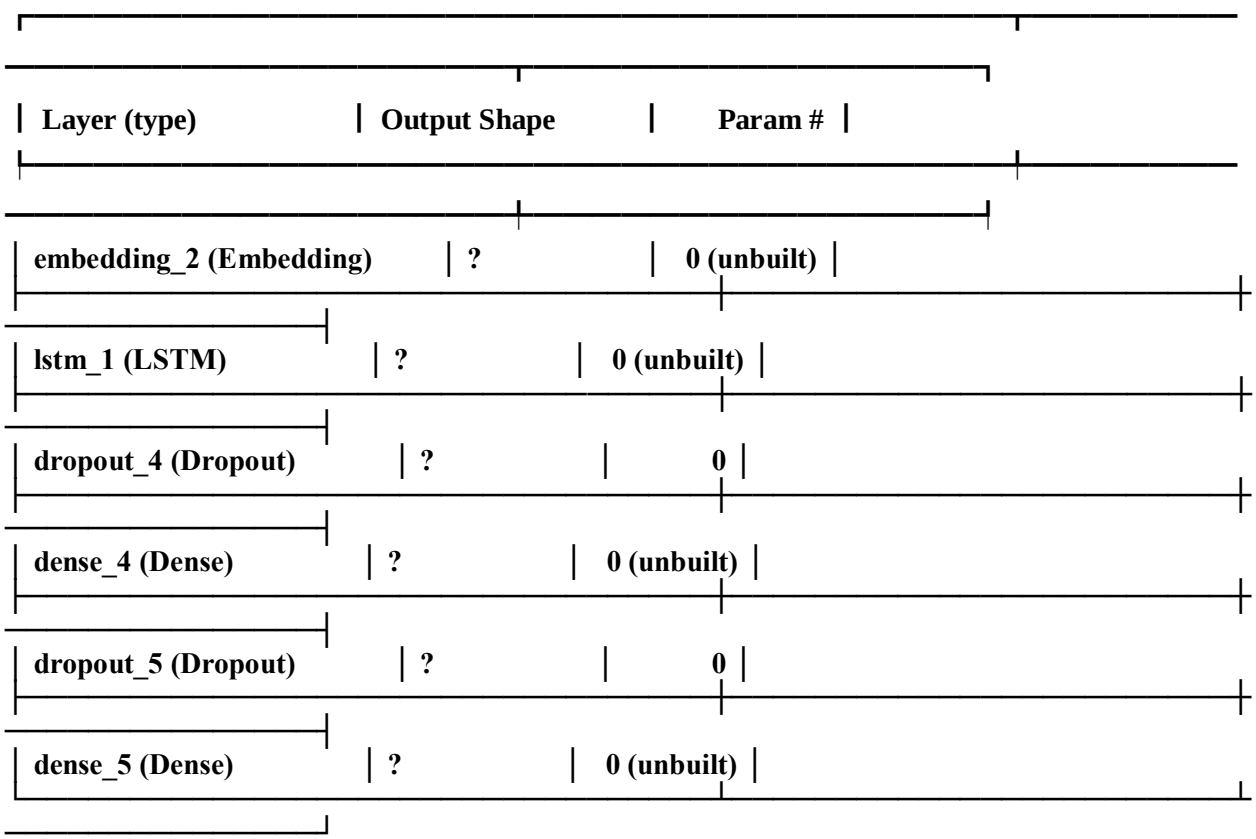
==========

Model Summary:

/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.

warnings.warn(

Model: "sequential_2"



Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	?	0 (unbuilt)
lstm_1 (LSTM)	?	0 (unbuilt)
dropout_4 (Dropout)	?	0
dense_4 (Dense)	?	0 (unbuilt)
dropout_5 (Dropout)	?	0
dense_5 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

Epoch 1/5
57/57 ━━━━━━━━━━━━━━━━━━━━ 53s 885ms/step - accuracy: 0.3620 - loss: 2.6895 - val_accuracy: 0.4385 - val_loss: 2.1387

Epoch 2/5
57/57 ━━━━━━━━━━━━━━━━━━━━ 50s 878ms/step - accuracy: 0.4582 - loss: 2.0193 - val_accuracy: 0.4752 - val_loss: 1.8542

Epoch 3/5
57/57 ━━━━━━━━━━━━━━━━━━━━ 82s 873ms/step - accuracy: 0.4799 - loss: 1.8646 - val_accuracy: 0.5320 - val_loss: 1.7380

Epoch 4/5
57/57 ━━━━━━━━━━━━━━━━━━━━ 49s 868ms/step - accuracy: 0.5338 - loss: 1.7355 - val_accuracy: 0.5381 - val_loss: 1.7145

Epoch 5/5
57/57 ━━━━━━━━━━━━━━━━━━━━ 82s 875ms/step - accuracy: 0.5557 - loss: 1.6208 - val_accuracy: 0.5799 - val_loss: 1.6360

LSTM RESULTS
Accuracy : 0.5726
Precision: 0.5178
Recall : 0.5726

F1 Score : 0.5323
Training Time: 348.47 seconds
Parameters: 1422830

Classification Report:

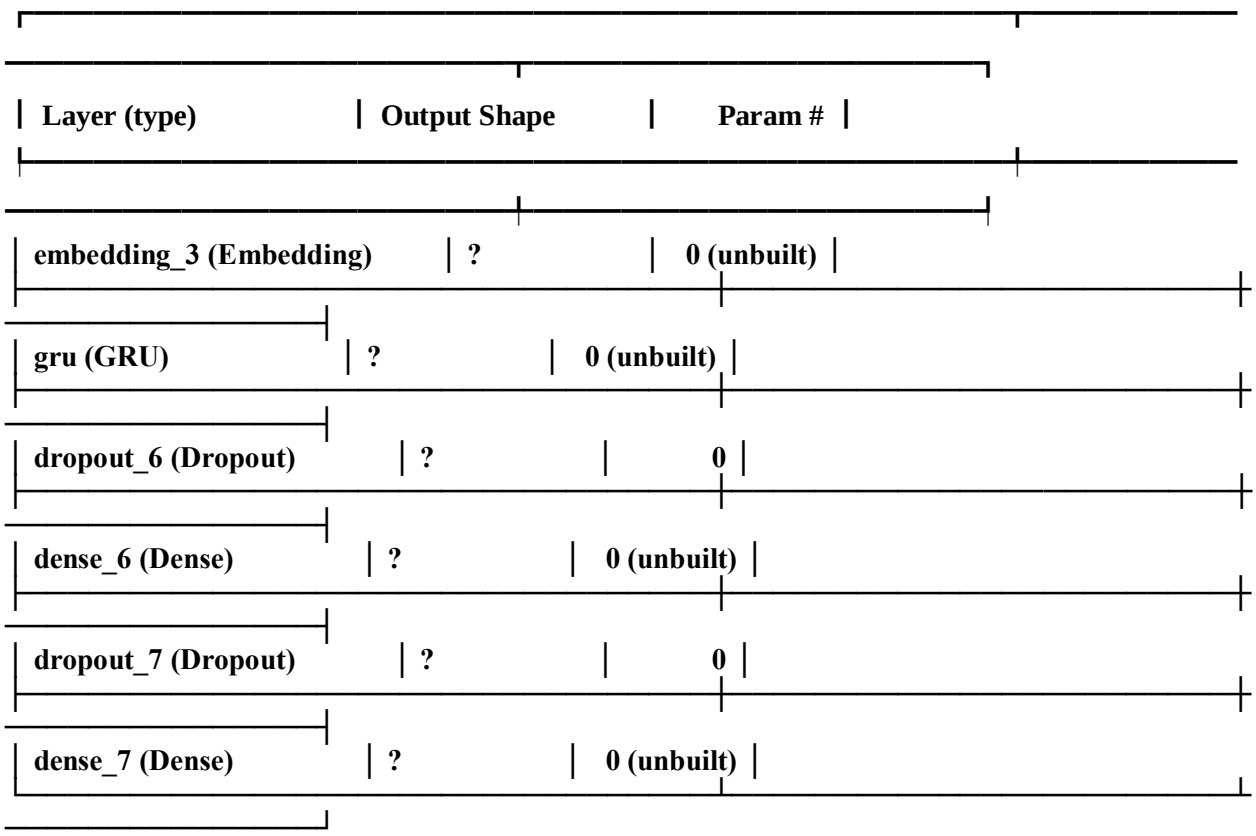
	precision	recall	f1-score	support
0	0.00	0.00	0.00	12
1	0.12	0.12	0.12	105
2	0.00	0.00	0.00	20
3	0.91	0.93	0.92	813
4	0.77	0.86	0.81	474
5	0.00	0.00	0.00	5
6	1.00	0.14	0.25	14
7	0.00	0.00	0.00	3
8	0.00	0.00	0.00	38
9	0.00	0.00	0.00	25
10	0.00	0.00	0.00	30
11	0.07	0.12	0.09	83
12	0.00	0.00	0.00	13
13	0.00	0.00	0.00	37
14	0.00	0.00	0.00	2
15	0.00	0.00	0.00	9
16	0.00	0.00	0.00	99
17	0.00	0.00	0.00	12
18	0.00	0.00	0.00	20
19	0.17	0.70	0.27	133
20	0.00	0.00	0.00	70
21	0.00	0.00	0.00	27
22	0.00	0.00	0.00	7
23	0.00	0.00	0.00	12
24	0.00	0.00	0.00	19
25	0.08	0.23	0.12	31
26	0.00	0.00	0.00	8
27	0.00	0.00	0.00	4
28	0.00	0.00	0.00	10
29	0.00	0.00	0.00	4
30	0.00	0.00	0.00	12
31	0.00	0.00	0.00	13
32	0.00	0.00	0.00	10
33	0.00	0.00	0.00	5
34	0.00	0.00	0.00	7
35	0.00	0.00	0.00	6
36	0.00	0.00	0.00	11
37	0.00	0.00	0.00	2
38	0.00	0.00	0.00	3
39	0.00	0.00	0.00	5
40	0.00	0.00	0.00	10
41	0.00	0.00	0.00	8
42	0.00	0.00	0.00	3
43	0.00	0.00	0.00	6

```
          44       0.00      0.00      0.00         5
          45       0.00      0.00      0.00         1

    accuracy                           0.57      2246
   macro avg       0.07      0.07      0.06      2246
weighted avg       0.52      0.57      0.53      2246


================================================================================
TRAINING: GRU
================================================================================

Model Summary:
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
Model: "sequential_3"
```



Layer (type)	Output Shape	Param #
embedding_3 (Embedding)	?	0 (unbuilt)
gru (GRU)	?	0 (unbuilt)
dropout_6 (Dropout)	?	0
dense_6 (Dense)	?	0 (unbuilt)
dropout_7 (Dropout)	?	0
dense_7 (Dense)	?	0 (unbuilt)

```
 Total params: 0 (0.00 B)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 0 (0.00 B)
Epoch 1/5
57/57 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 52s 844ms/step - accuracy: 0.3447 - loss: 2.7332 - val_accuracy: 0.4669 - val_loss: 2.1056
Epoch 2/5
```

57/57 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 48s 848ms/step - accuracy: 0.4690 - loss: 2.0252 - val_accuracy: 0.4808 - val_loss: 1.9636
Epoch 3/5
57/57 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 49s 862ms/step - accuracy: 0.5072 - loss: 1.8270 - val_accuracy: 0.5264 - val_loss: 1.8112
Epoch 4/5
57/57 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 47s 814ms/step - accuracy: 0.5450 - loss: 1.6742 - val_accuracy: 0.5231 - val_loss: 1.8339
Epoch 5/5
57/57 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 48s 839ms/step - accuracy: 0.5843 - loss: 1.5760 - val_accuracy: 0.5726 - val_loss: 1.7151

GRU RESULTS
Accuracy : 0.5677
Precision: 0.5215
Recall : 0.5677
F1 Score : 0.528
Training Time: 243.84 seconds
Parameters: 1390318

Classification Report:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	12
1	0.18	0.68	0.28	105
2	0.00	0.00	0.00	20
3	0.94	0.93	0.93	813
4	0.77	0.76	0.76	474
5	0.00	0.00	0.00	5
6	0.00	0.00	0.00	14
7	0.00	0.00	0.00	3
8	0.00	0.00	0.00	38
9	0.00	0.00	0.00	25
10	0.00	0.00	0.00	30
11	0.00	0.00	0.00	83
12	0.00	0.00	0.00	13
13	0.00	0.00	0.00	37
14	0.00	0.00	0.00	2
15	0.00	0.00	0.00	9
16	0.00	0.00	0.00	99
17	0.00	0.00	0.00	12
18	0.00	0.00	0.00	20
19	0.16	0.69	0.26	133
20	0.00	0.00	0.00	70
21	0.00	0.00	0.00	27
22	0.00	0.00	0.00	7
23	0.00	0.00	0.00	12
24	0.00	0.00	0.00	19
25	0.00	0.00	0.00	31

26	0.00	0.00	0.00	8
27	0.00	0.00	0.00	4
28	0.00	0.00	0.00	10
29	0.00	0.00	0.00	4
30	0.00	0.00	0.00	12
31	0.00	0.00	0.00	13
32	0.00	0.00	0.00	10
33	0.00	0.00	0.00	5
34	0.00	0.00	0.00	7
35	0.00	0.00	0.00	6
36	0.00	0.00	0.00	11
37	0.00	0.00	0.00	2
38	0.00	0.00	0.00	3
39	0.00	0.00	0.00	5
40	0.00	0.00	0.00	10
41	0.00	0.00	0.00	8
42	0.00	0.00	0.00	3
43	0.00	0.00	0.00	6
44	0.00	0.00	0.00	5
45	0.00	0.00	0.00	1
accuracy			0.57	2246
macro avg	0.04	0.07	0.05	2246
weighted avg	0.52	0.57	0.53	2246

==========

FINAL MODEL COMPARISON

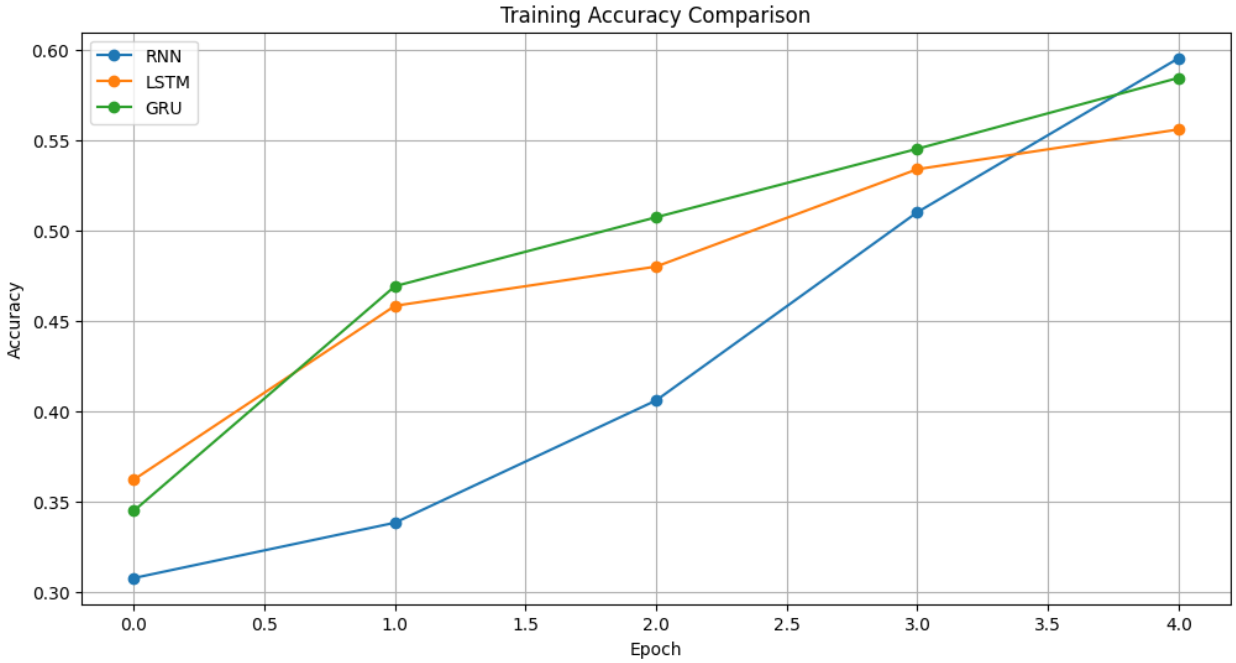
==========

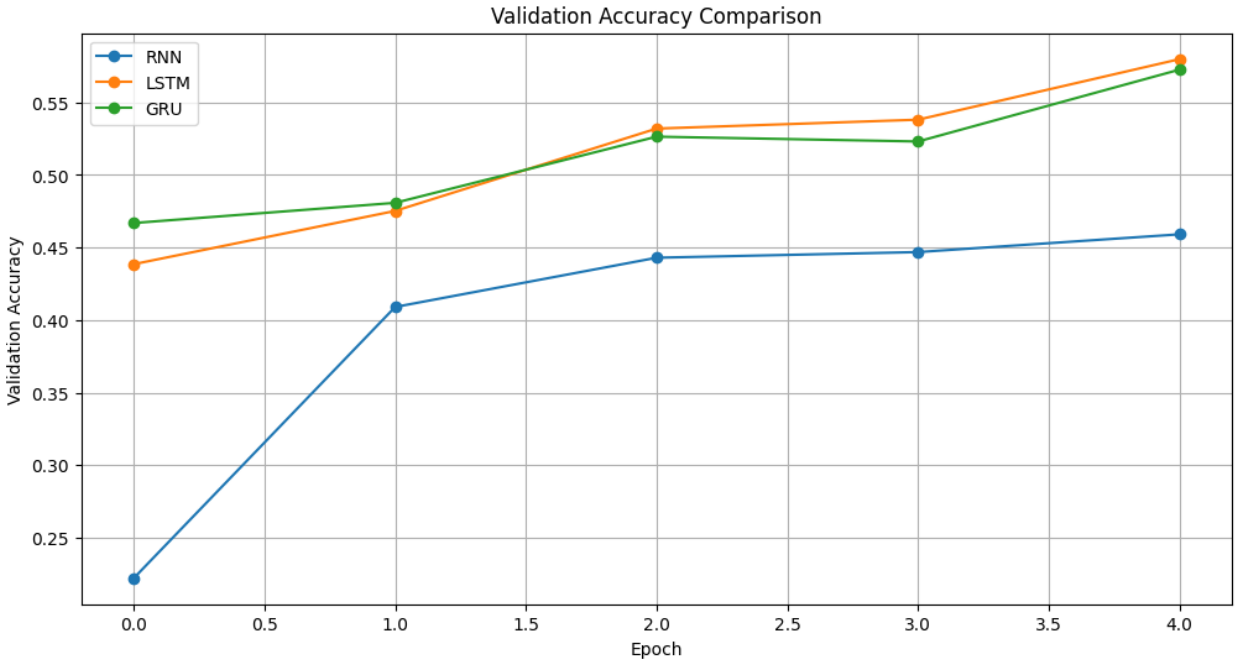
	Model	Accuracy	Precision	Recall	F1 Score	Training Time (sec)	\
0	RNN	0.445236	0.412647	0.445236	0.399313	128.938954	
1	LSTM	0.572573	0.517804	0.572573	0.532325	348.466768	
2	GRU	0.567676	0.521543	0.567676	0.527975	243.844408	

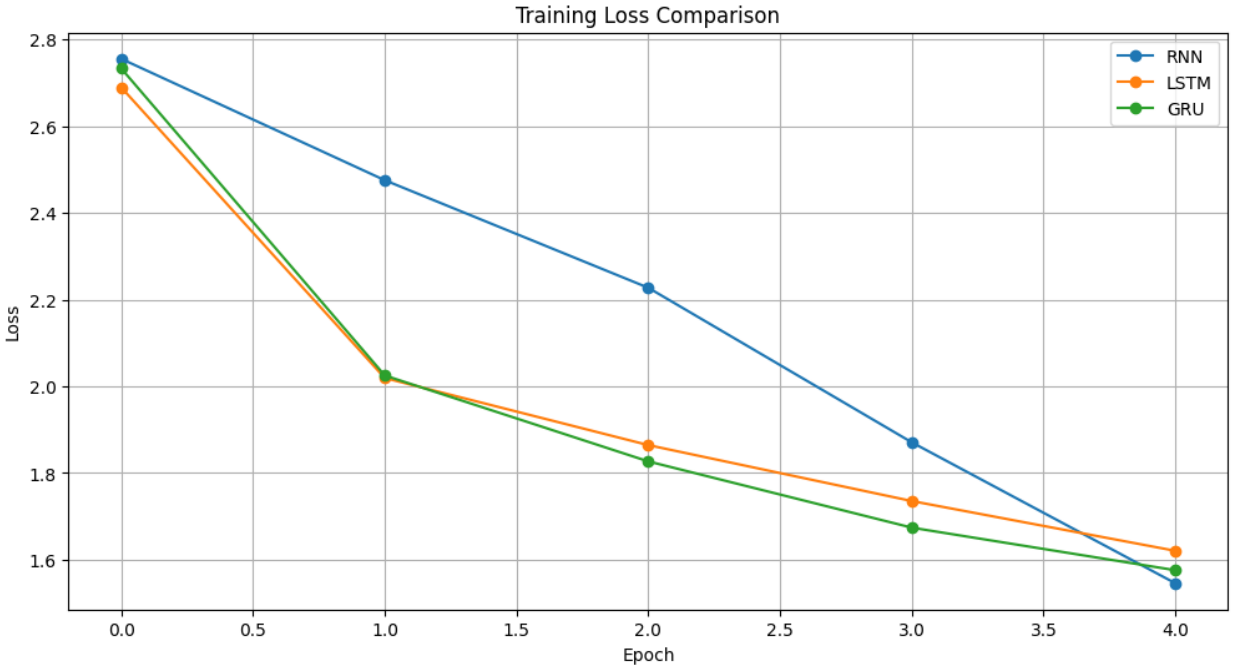
	Parameters
0	1324142
1	1422830
2	1390318

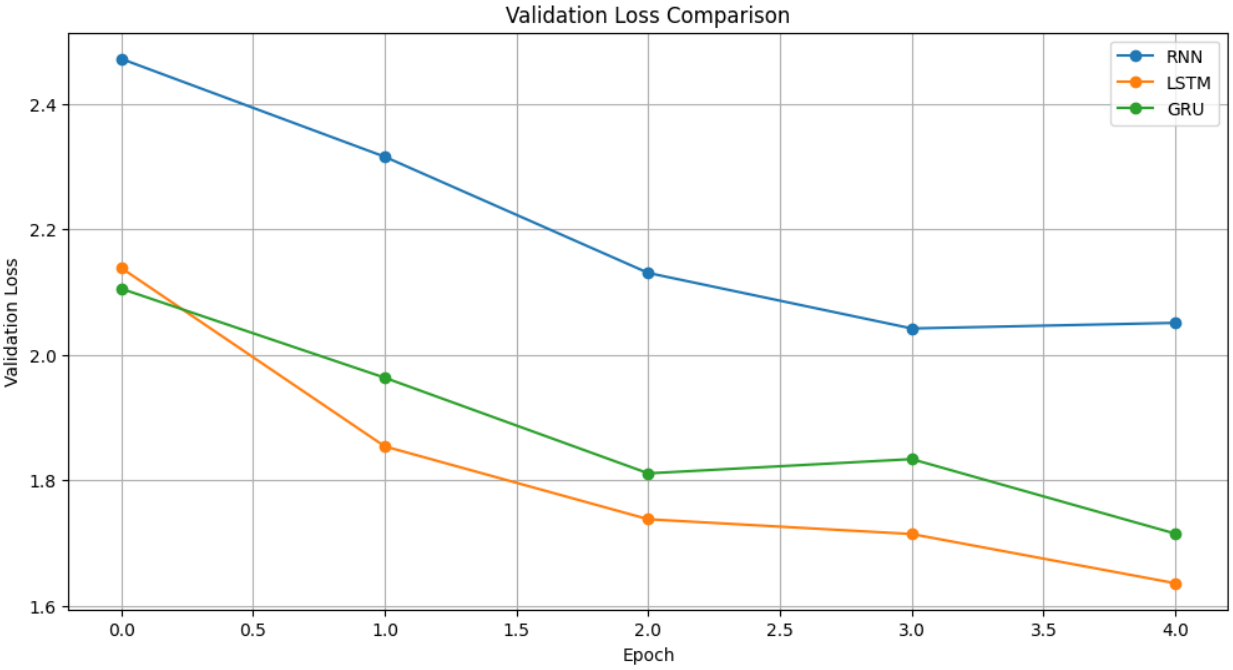
Formatted Results:

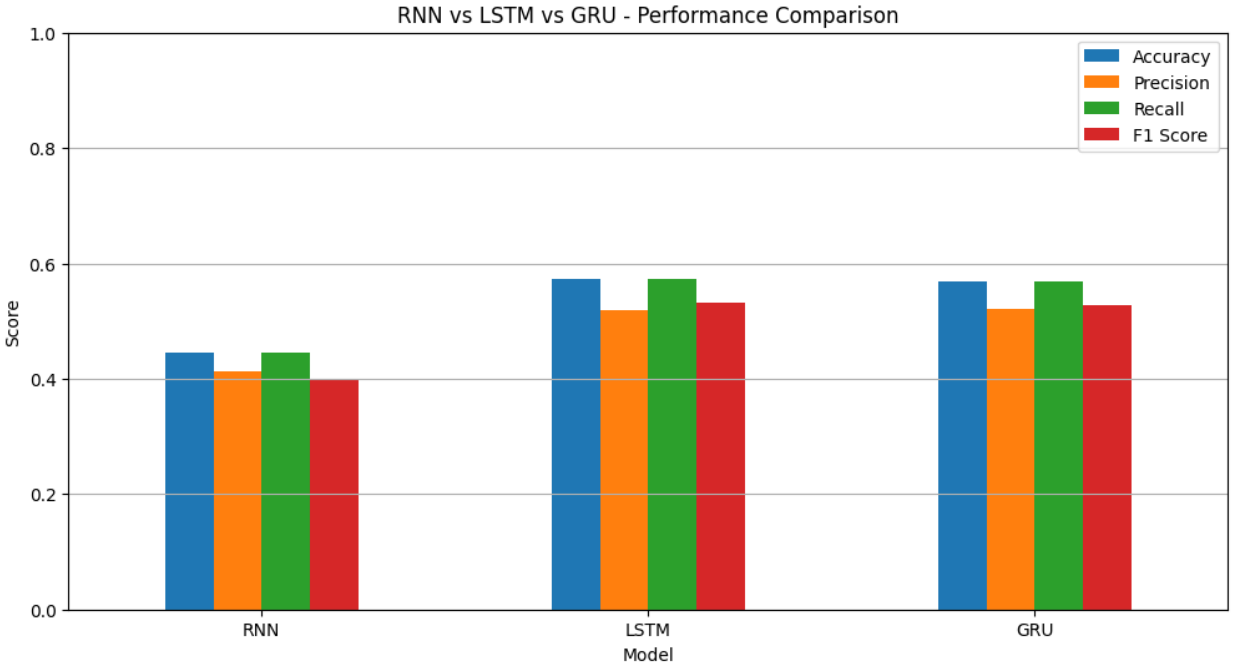
	Model	Accuracy	Precision	Recall	F1 Score	Training Time (sec)	Parameters
0	RNN	0.4452	0.4126	0.4452	0.3993	128.94	1324142
1	LSTM	0.5726	0.5178	0.5726	0.5323	348.47	1422830
2	GRU	0.5677	0.5215	0.5677	0.5280	243.84	1390318

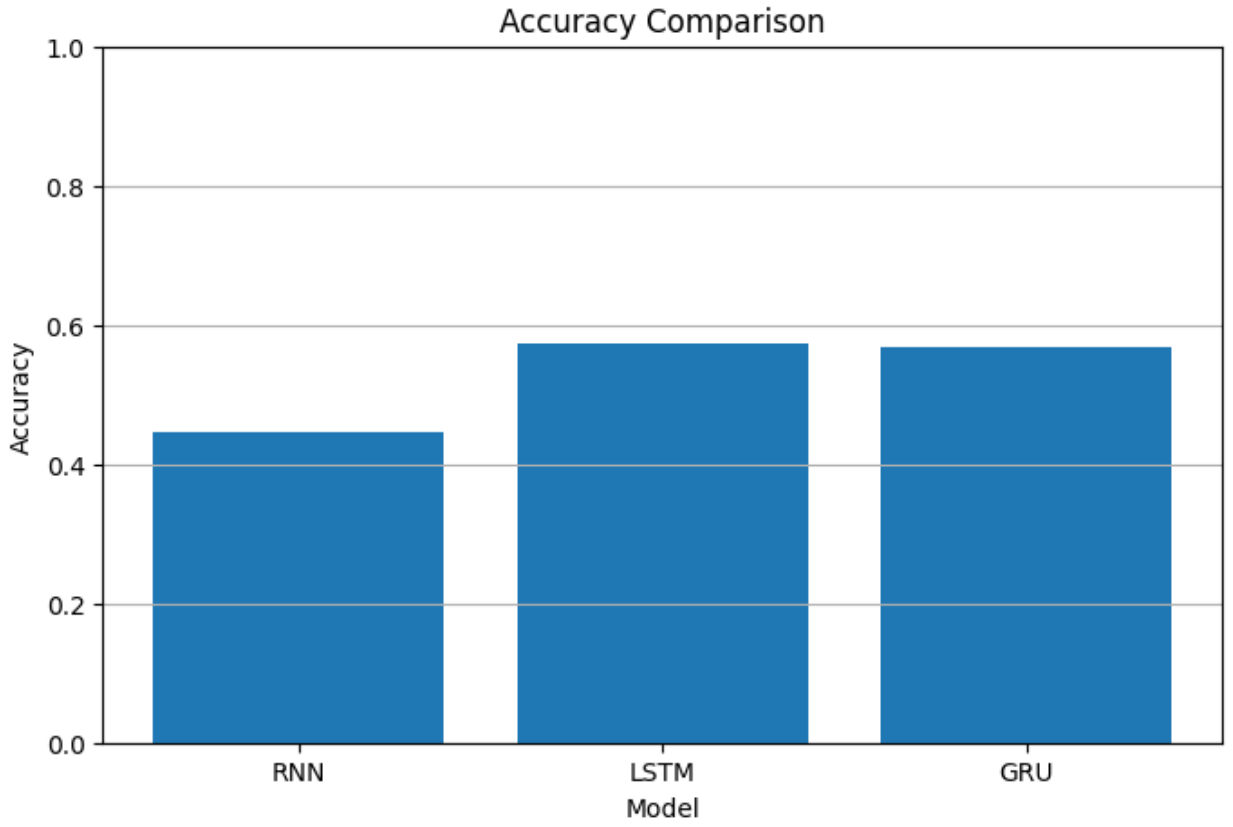

Training Accuracy Comparison
RNN
LSTM
GRU
Accuracy
Epoch
0.60
0.55
0.50
0.45
0.40
0.35
0.30
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0

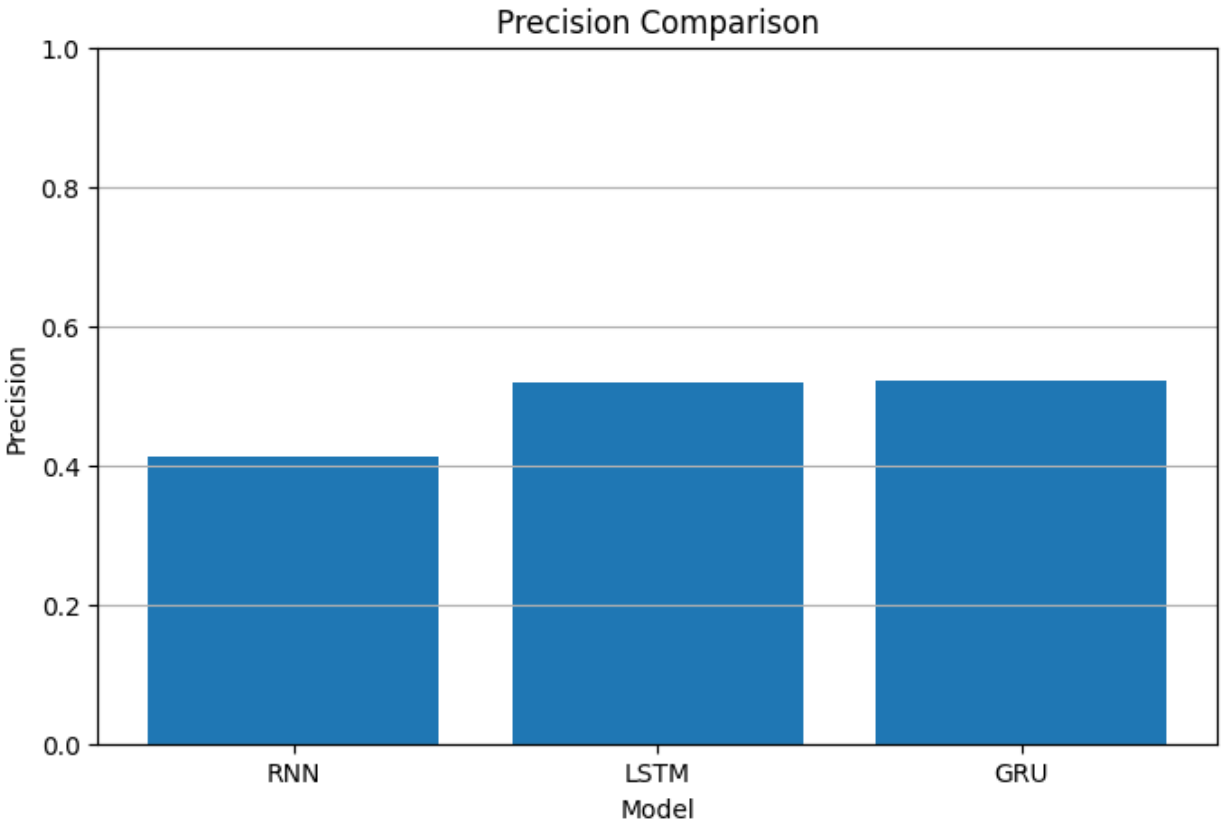

Validation Accuracy Comparison
RNN
LSTM
GRU
Validation Accuracy
Epoch
0.55
0.50
0.45
0.40
0.35
0.30
0.25
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0

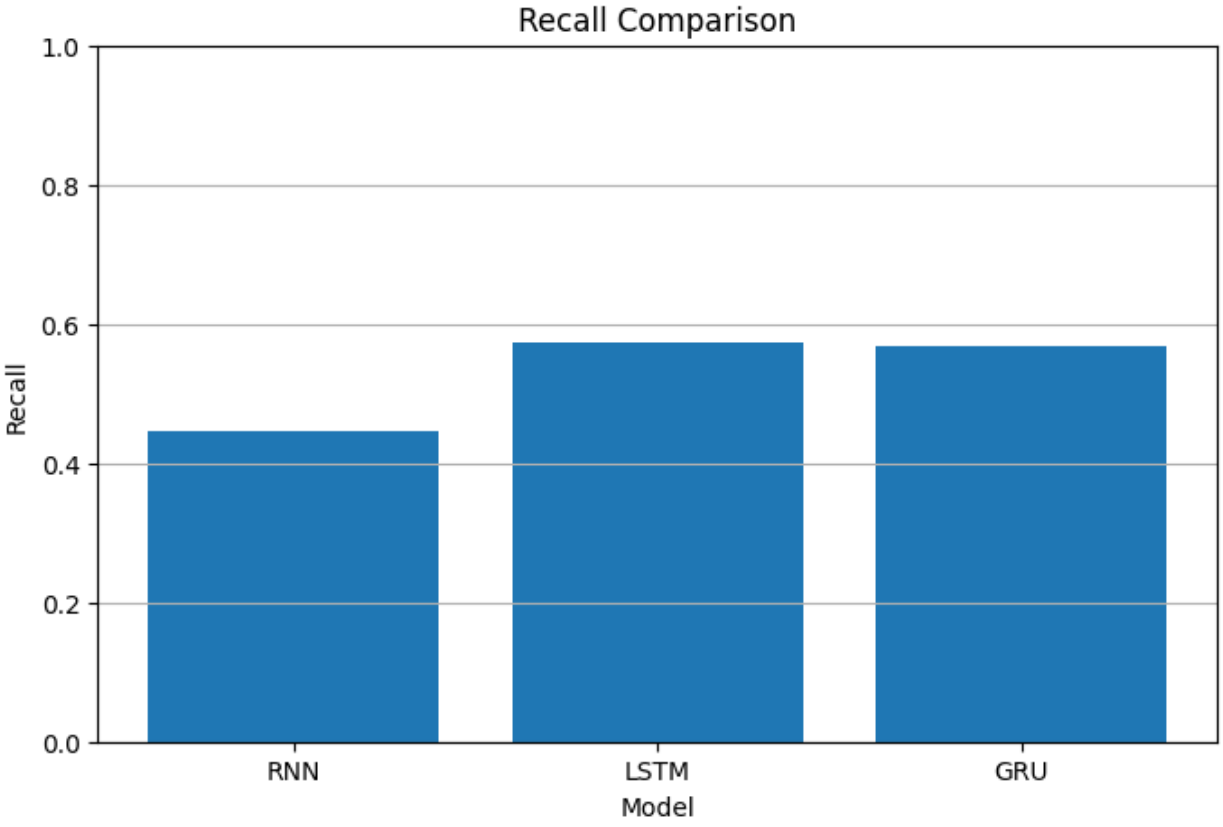

Training Loss Comparison
RNN
LSTM
GRU
Loss
Epoch
2.8
2.6
2.4
2.2
2.0
1.8
1.6
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0

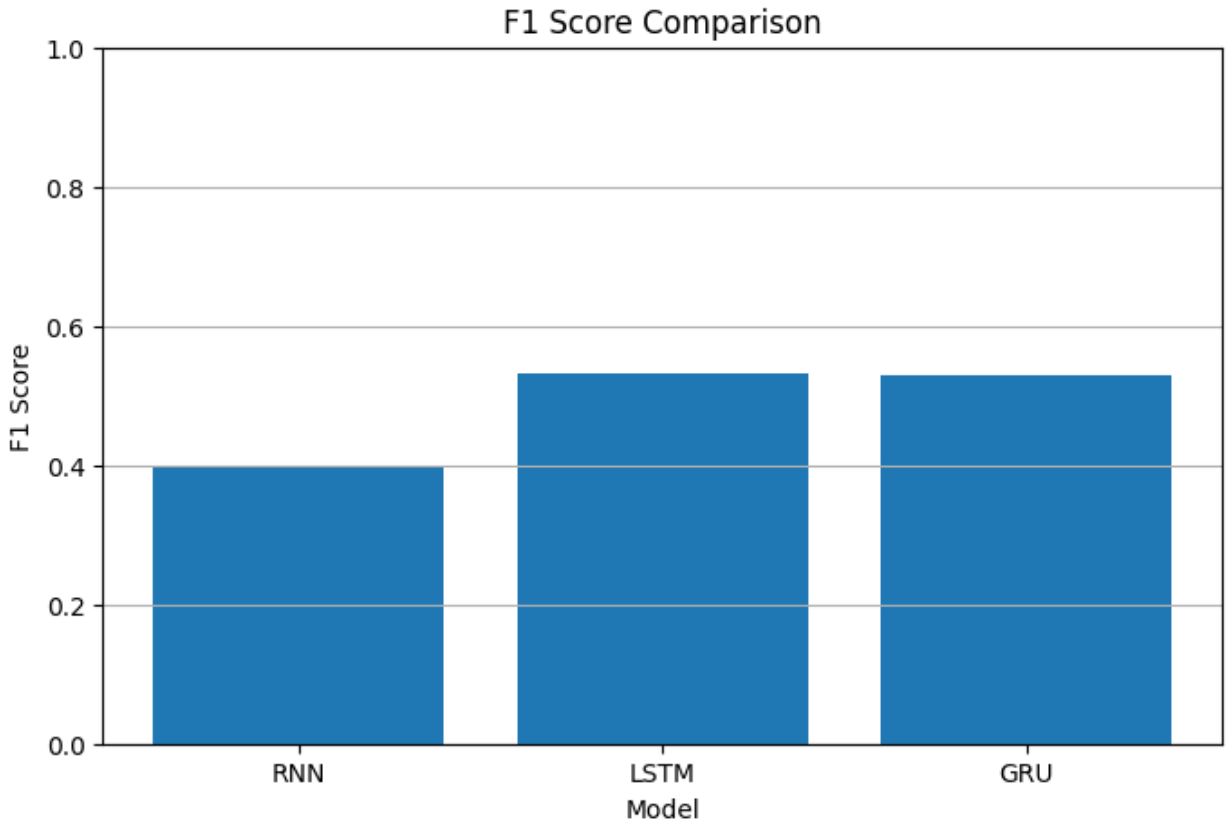

Validation Loss Comparison
RNN
LSTM
GRU
Validation Loss
Epoch
2.4
2.2
2.0
1.8
1.6
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0

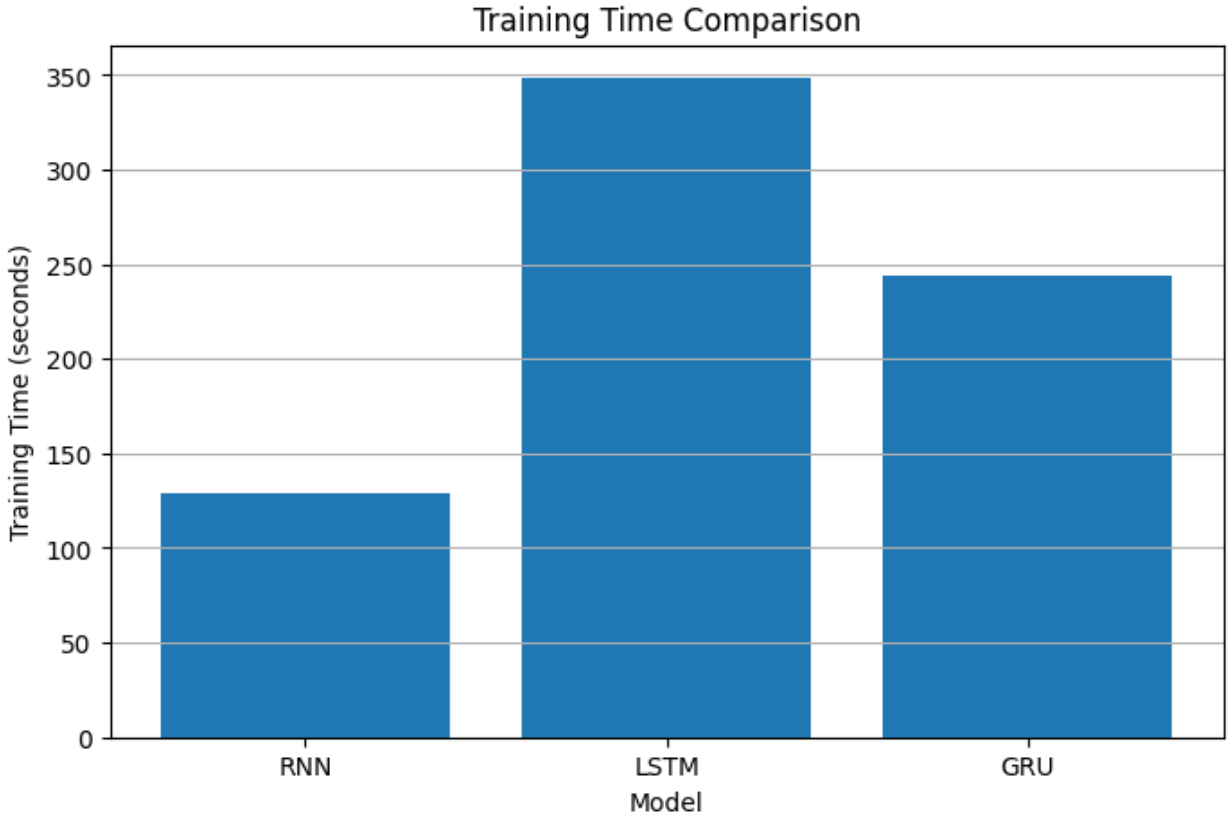

RNN vs LSTM vs GRU - Performance Comparison
Accuracy
Precision
Recall
F1 Score
Score
1.0
0.8
0.6
0.4
0.2
0.0
RNN
LSTM
GRU
Model

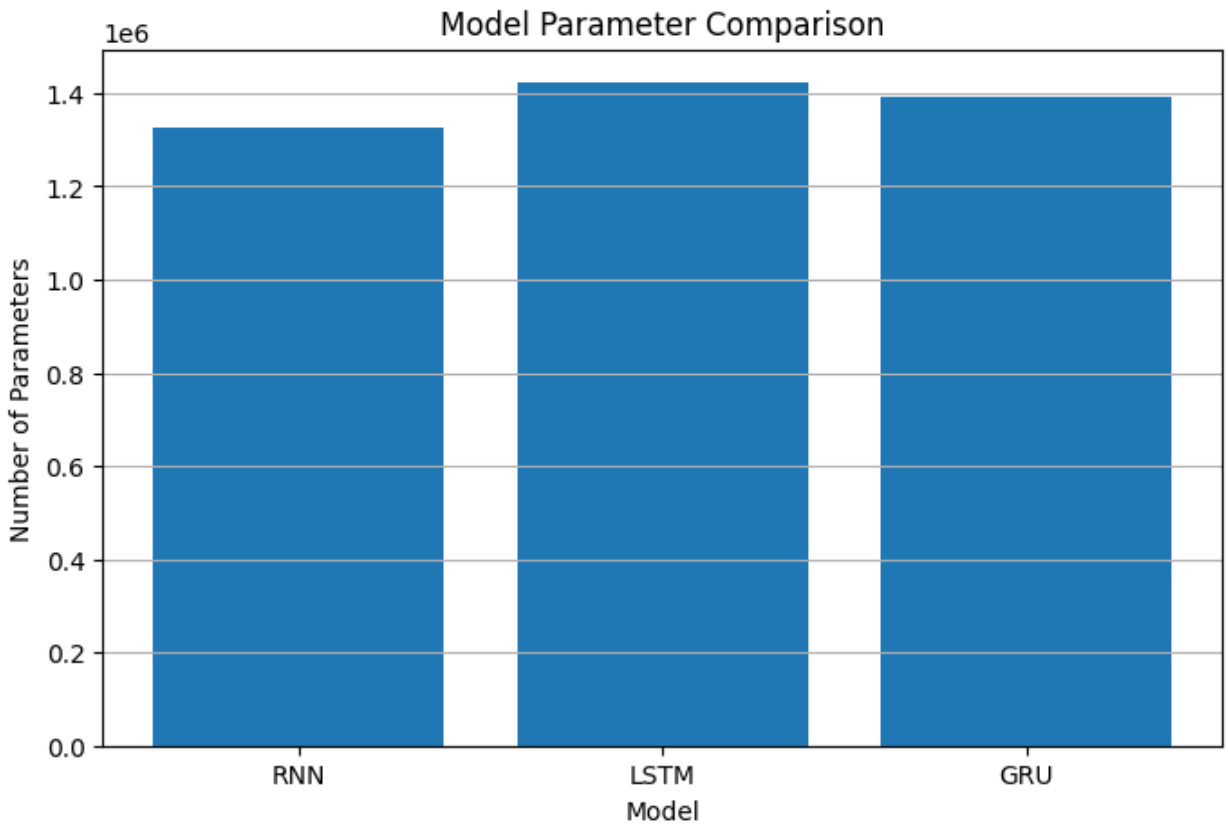

Accuracy Comparison
Accuracy
1.0
0.8
0.6
0.4
0.2
0.0
RNN
LSTM
GRU
Model

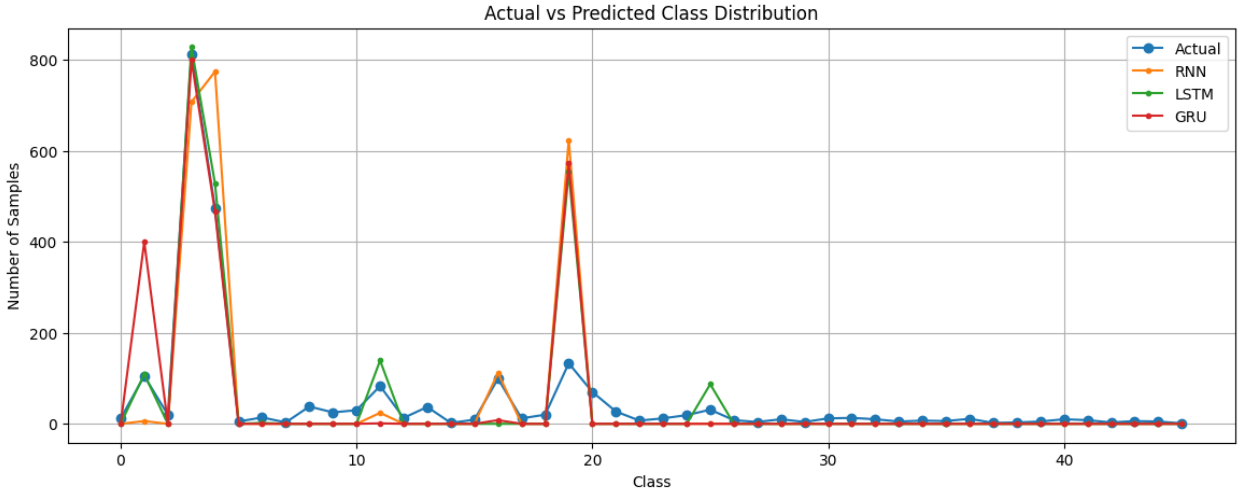

Precision Comparison
1.0
0.8
0.6
0.4
0.2
0.0
Precision
RNN
LSTM
GRU
Model


Recall Comparison
1.0
0.8
0.6
0.4
0.2
0.0
Recall
RNN
LSTM
GRU
Model


F1 Score Comparison
F1 Score
1.0
0.8
0.6
0.4
0.2
0.0
RNN
LSTM
GRU
Model


Training Time Comparison
Training Time (seconds)
350
300
250
200
150
100
50
0
RNN
LSTM
GRU
Model





BEST PERFORMANCE BY METRIC
Accuracy: LSTM (0.5726)
Precision: GRU (0.5215)
Recall: LSTM (0.5726)
F1 Score: LSTM (0.5323)